UNIT-7
The Transport Layer

Department of Computer Science and Applications,
MITWPU, Pune

Prepared by:
Dr. Suvarna Sharma

Functions of Transport Layer

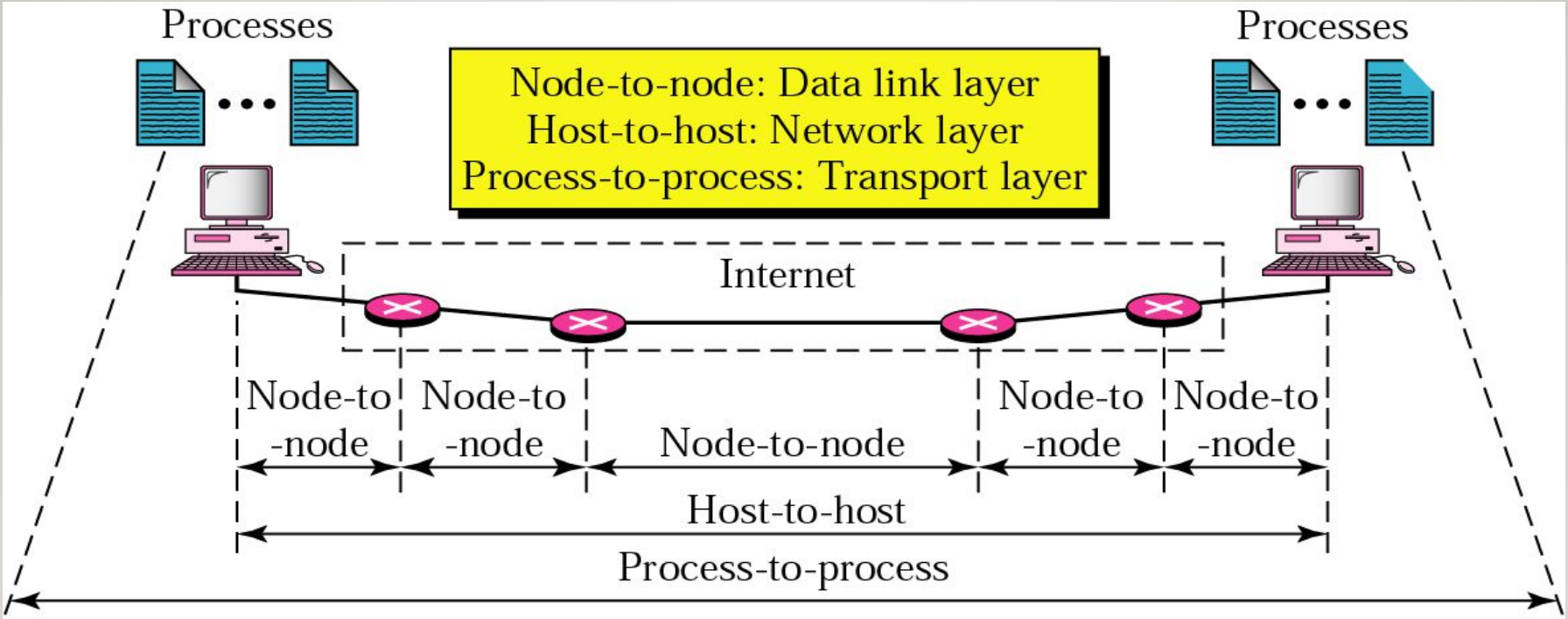
- **Process-to-Process Delivery:** Ensures data is delivered between the correct sender and receiver processes.
- **End-to-End Connection:** Establishes and maintains a logical connection between communicating devices.
- **Multiplexing and Demultiplexing:** Allows multiple applications to send and receive data simultaneously using port numbers.
- **Data Integrity and Error Correction:** Detects and corrects errors to ensure reliable data transmission.
- **Congestion Control:** Controls data flow to prevent network congestion.

PROCESS-TO-PROCESS DELIVERY

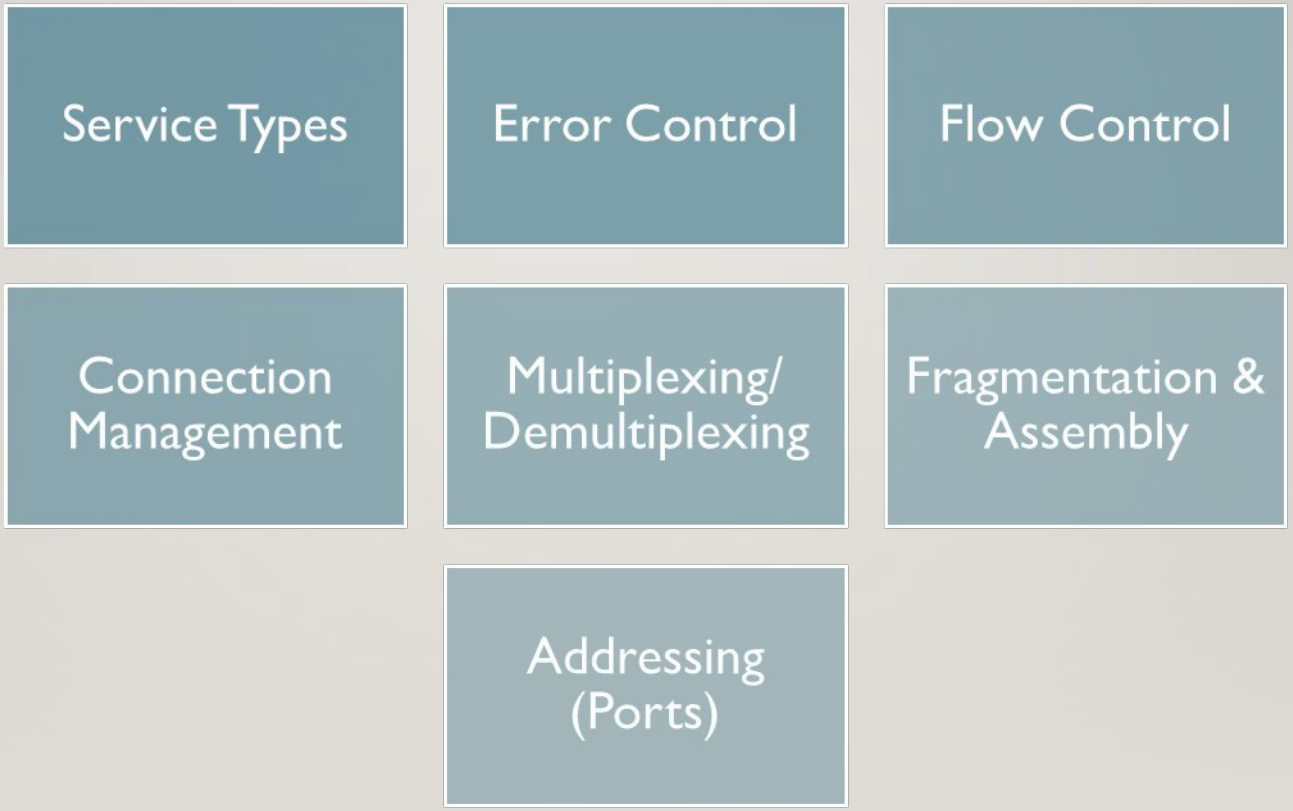
- The data link layer is responsible for delivery of frames between two neighboring nodes over a link. This is called ***node-to-node delivery***.
- The network layer is responsible for delivery of datagrams between two hosts. This is called ***host-to-host delivery***.
- *Communication* on the Internet is not defined as the exchange of data between two nodes or between two hosts. Real communication takes place between two processes (application programs). We need process-to-process delivery. However, at any moment, several processes may be running on the source host and several on the destination host.
- To complete the delivery, we need a mechanism to deliver data from one of these processes running on the source host to the corresponding process running on the destination host.
- The transport layer is responsible for process-to-process delivery of

  a packet, part of a message, from one process to another.

Types of data deliveries



Elements of Transport Protocols



SERVICE TYPES

A transport layer protocol can either be connectionless or connection-oriented.

- *Connectionless Service*

  In a connectionless service, the packets are sent from one party to another with no need for connection establishment or connection release. The packets are not numbered; they may be delayed or lost or may arrive out of sequence. There is no acknowledgment either. UDP is connectionless.

- *Connection-Oriented Service*

  In a connection-oriented service, a connection is first established between the sender and the receiver. Data are transferred. At the end, the connection is released. TCP and SCTP are connection-oriented protocols.

ERROR CONTROL

Purpose: Ensures reliable delivery of data by detecting errors and retransmitting lost or corrupted segments.

Key Points:

- Acknowledgments (ACKs):

  Definition: Sent by the receiver to confirm successful receipt of data segments.

  Timeout Mechanism: If ACKs are not received within a specified timeout period, the sender assumes the segment is lost and retransmits it.

- Selective Repeat and Go-Back-N ARQ:

  Definitions:

  Selective Repeat: Retransmits only the lost segments, minimizing retransmission overhead.

  Go-Back-N: Retransmits all segments from the lost segment onward, simplifying the implementation but potentially wasting network resources.

FLOW CONTROL

Purpose: Regulates the rate of data transmission between sender and receiver to prevent overflow and ensure efficient utilization of network resources.

Key Points:

- Sliding Window Protocol:

Definition: Allows multiple data segments to be in transit simultaneously between sender and receiver.

Sender Side: Maintains a congestion window to control the number of unacknowledged segments transmitted at any time.

Receiver Side: Adjusts the receiver window size to inform the sender about the amount of data it can accept..

- Buffering:

Definitions:

Selective Repeat: Retransmits only the lost segments, minimizing retransmission overhead.

Go-Back-N: Retransmits all segments from the lost segment onward, simplifying the implementation but potentially wasting network resources.

Connection Establishment

Establishing a connection sounds easy, but it is actually surprisingly tricky. At first glance, it would seem sufficient for one transport entity to just send a CONNECTION REQUEST segment to the destination and wait for a CONNECTION ACCEPTED reply. The problem occurs when the network can lose, delay, corrupt, and duplicate packets.

The Three-Way Handshake

The three-way handshake is a common method for establishing a connection:

1.Connection Request: The initiating host sends a connection request with a chosen sequence number.

2.Acknowledgment: The receiving host acknowledges the request and sends back its own initial sequence number.

3.Final Acknowledgment: The initiating host acknowledges the receiving host's sequence number in the first data segment it sends.

This method helps prevent issues with delayed duplicate packets, ensuring that only current connection requests are processed.

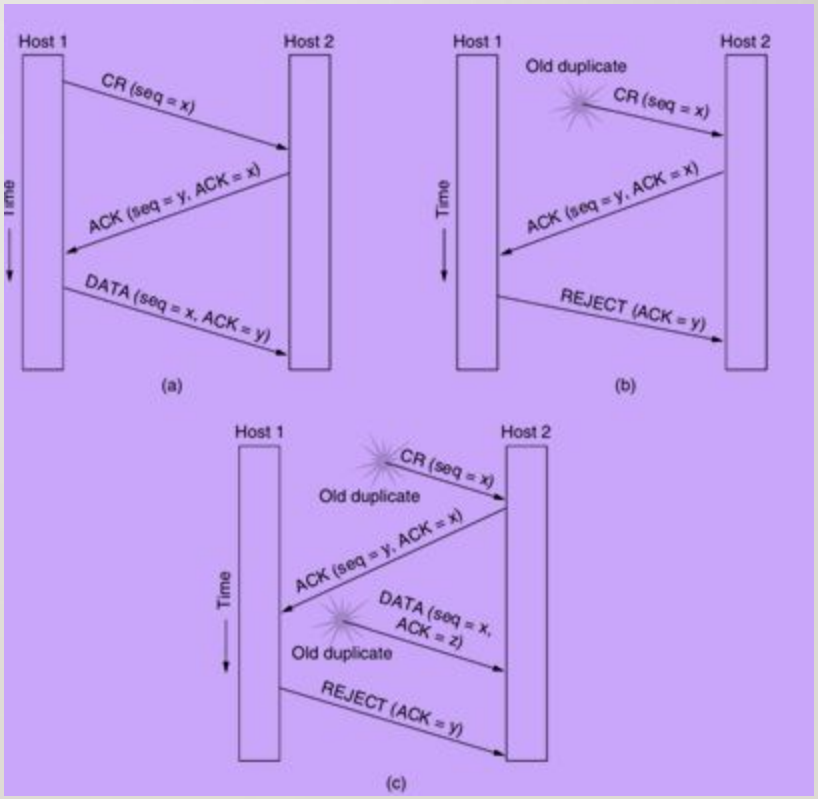
The Three-Way Handshake

Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes CONNECTION REQUEST.

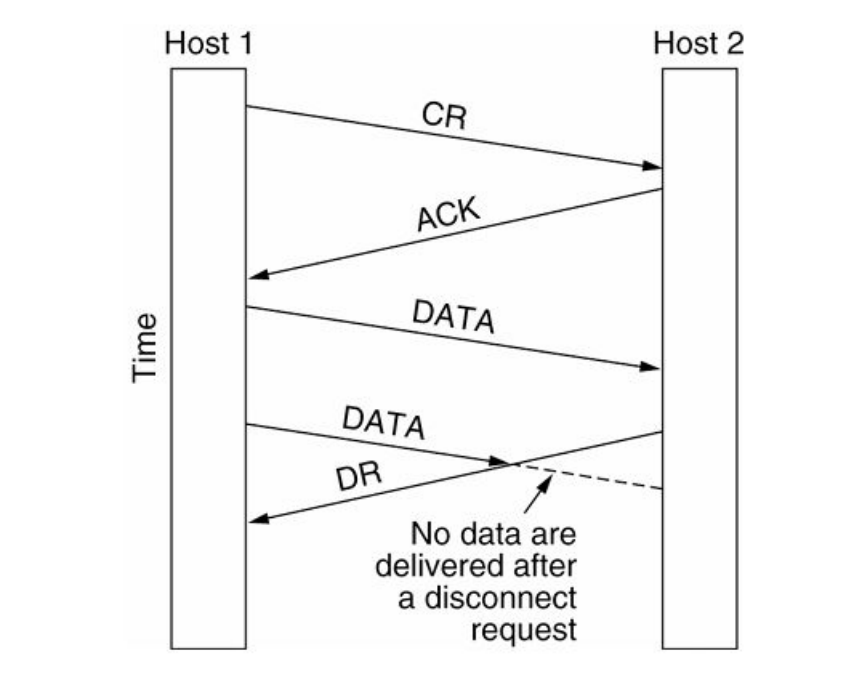
(a)Normal operation.

(b)Old duplicate CONNECTION REQUEST appearing out of nowhere.

(c)Duplicate CONNECTION REQUEST and duplicate ACK



Connection Release



Abrupt disconnection with loss of data

Connection Release

There are two styles of terminating a connection: asymmetric release and symmetric release

Asymmetric release is the way the telephone system works: when one party hangs up, the connection is broken.

Symmetric release treats the connection as two separate unidirectional connections and requires each one to be released separately Asymmetric release is abrupt and may result in data loss.

Consider the scenario of Fig. After the connection is established, host 1 sends a segment that arrives properly at host 2. Then host 1 sends another segment. Unfortunately, host 2 issues a DISCONNECT before the second segment arrives. The result is that the connection is released and data are lost.

Connection Release

Clearly, a more sophisticated release protocol is needed to avoid data loss. One way is to use symmetric release, in which each direction is released independently of the other one. Here, a host can continue to receive data even after it has sent a DISCONNECT segment.

Symmetric release does the job when each process has a fixed amount of data to send and clearly knows when it has sent it. One can envision a protocol in which host 1 says "I am done. Are you done too?" If host 2 responds: "I am done too. Goodbye, the connection can be safely released."

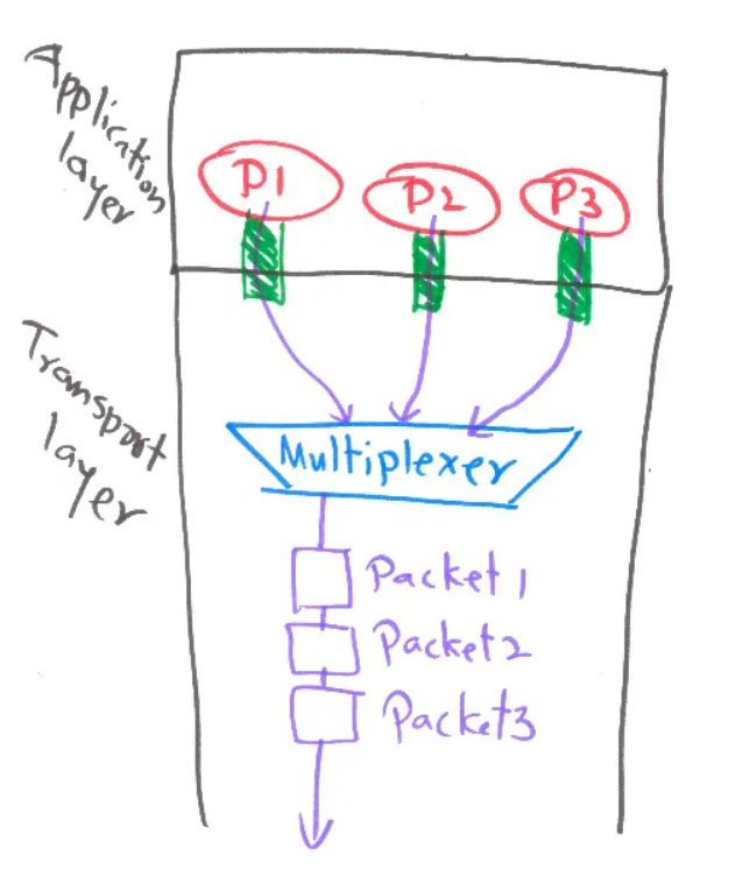
Multiplexing

Multiplexing is the process of collecting data from multiple application processes of the sender, enveloping that data with headers, and sending them as a whole to the intended receiver.

- In Multiplexing at the Transport Layer, data is collected from various application processes. These segments contain the source port number, destination port number, header files, and data.

- These segments are passed to the Network Layer which adds the source and destination IP address to create the datagram.

Multiplexing

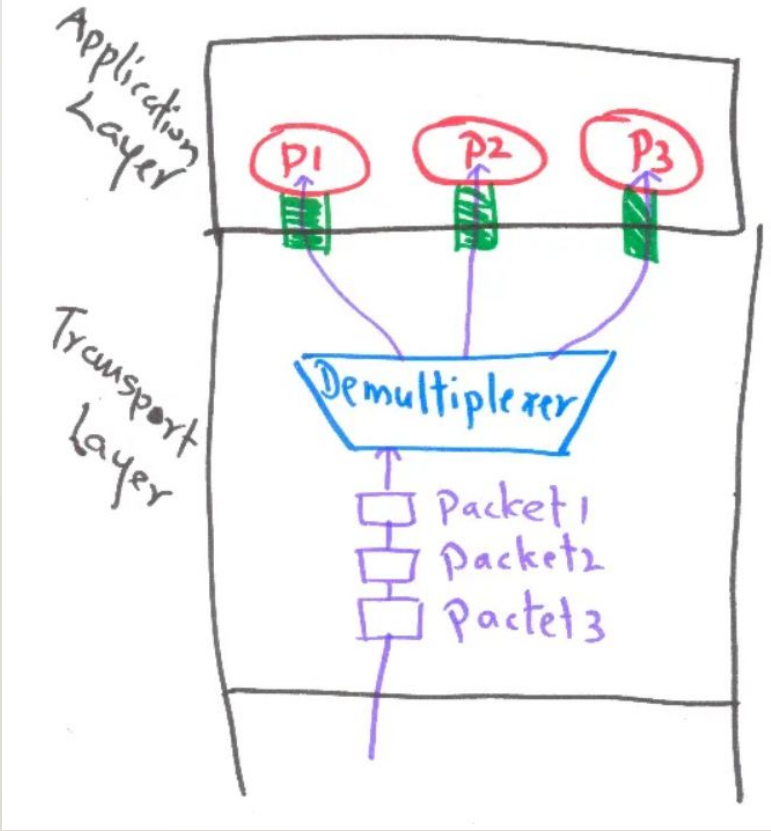


Demultiplexing

Delivering the received segments at the receiver side to the correct application layer processes is called demultiplexing.

- The destination host receives IP datagrams; each datagram has a source IP address and a destination IP address.
- Each datagram carries one transport layer segment with source and destination port numbers.
- The destination host uses the IP addresses and port numbers to direct the segment to the appropriate socket.

Demultiplexing



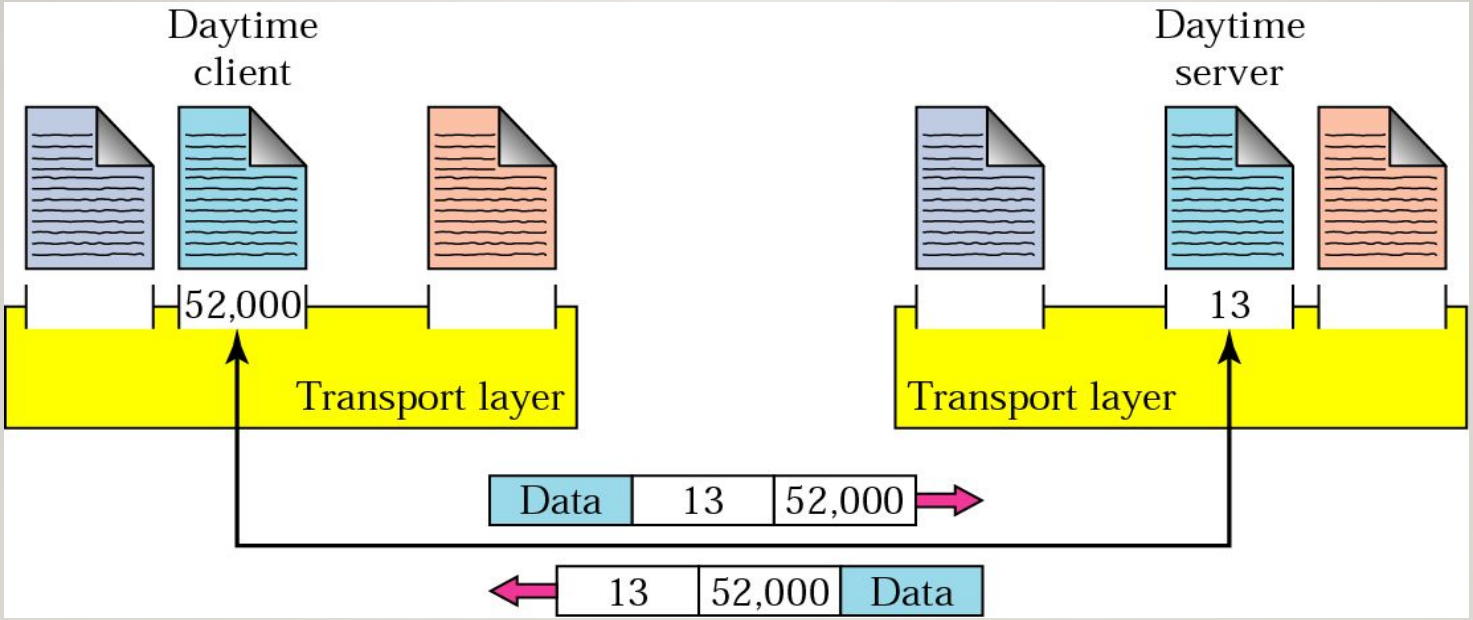
Fragmentation and Reassembly

When the transport layer receives a large message from the session layer, it breaks the message into smaller units depending upon requirements. This process is called **fragmentation**. Thereafter, it is passed to the network layer. Conversely, when the transport layer acts as the receiving process, it reorders the pieces of a message before reassembling them into a complete message.

Port Addressing

- In the Internet model, the port numbers are 16-bit integers between 0 and 65,535.
- The client program defines itself with a port number, chosen randomly by the transport layer software running on the client host. This is the ephemeral port number.
- The server process must also define itself with a port number. This port number, however, cannot be chosen randomly.
- The Internet has decided to use universal port numbers for servers; these are called well-known port numbers.

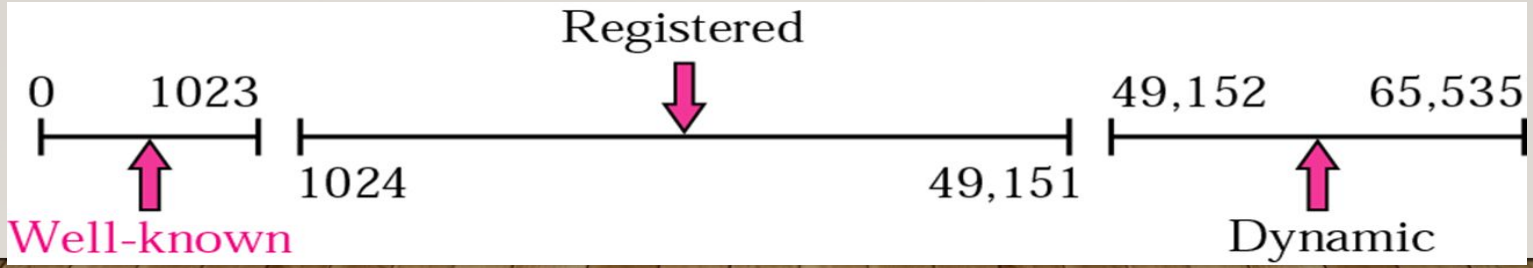
Port numbers



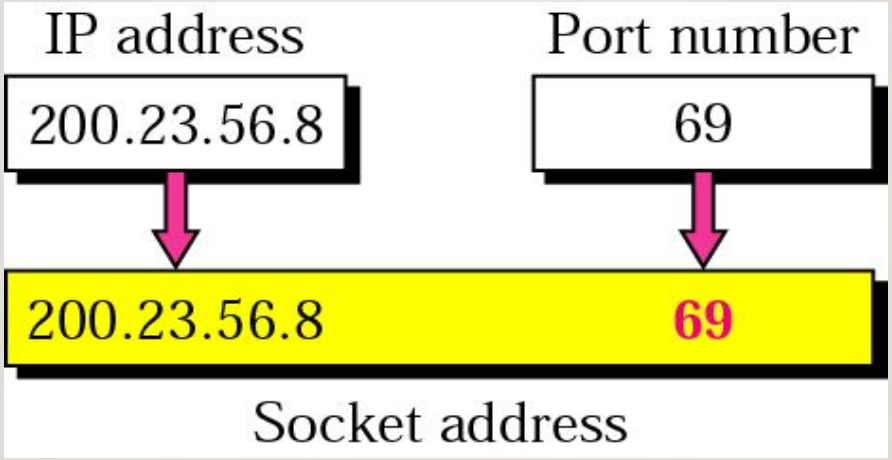
IANA Ranges

The IANA (**I**nternet **A**ssigned **N**umber **A**uthority) has divided the port numbers into three ranges: well known, registered, and dynamic (or private)

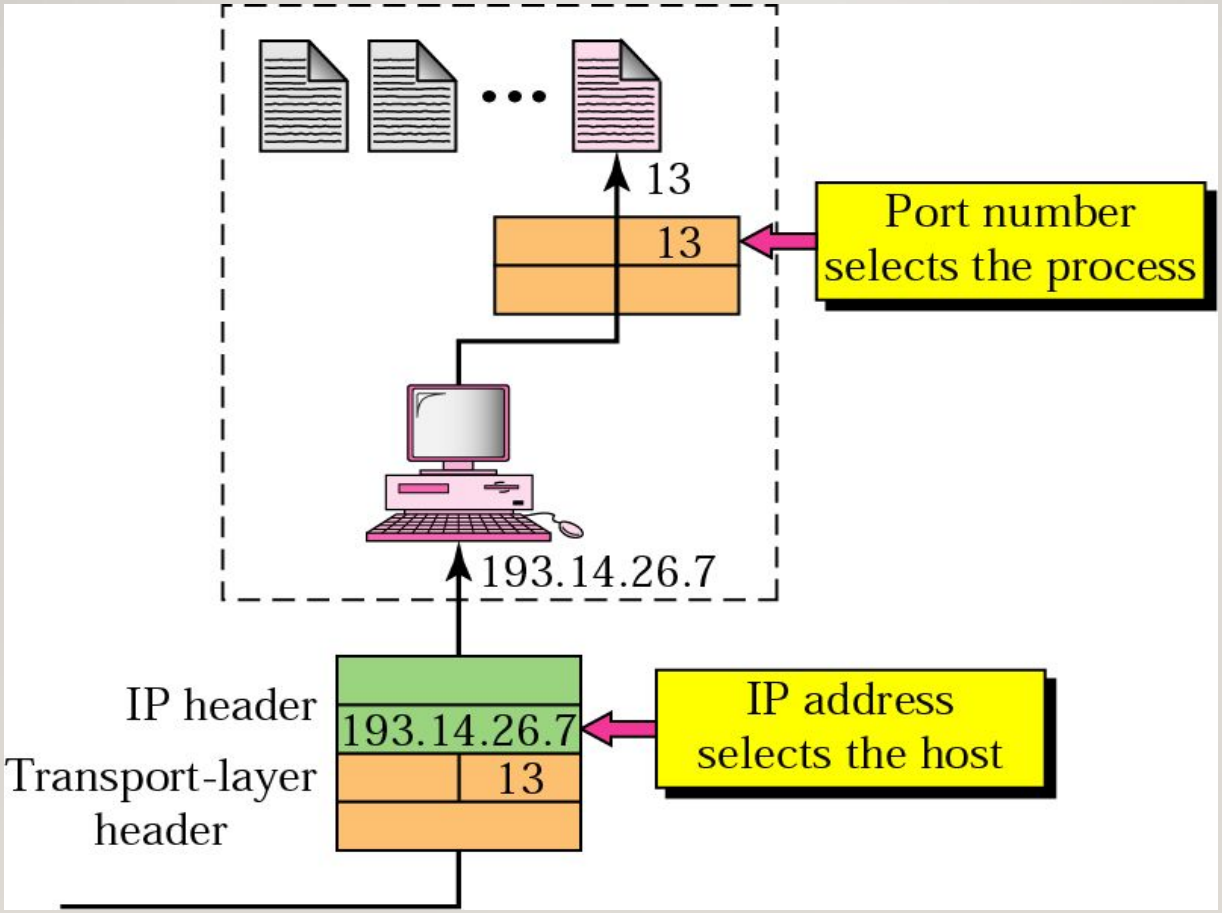
1. **Well-known ports.** The ports ranging from 0 to 1023 are assigned and controlled by IANA. These are the well-known ports.
2. **Registered ports.** The ports ranging from 1024 to 49,151 are not assigned or controlled by IANA. They can only be registered with IANA to prevent duplication.
3. **Dynamic ports.** The ports ranging from 49,152 to 65,535 are neither controlled nor registered. They can be used by any process. These are the ephemeral ports.



Socket address



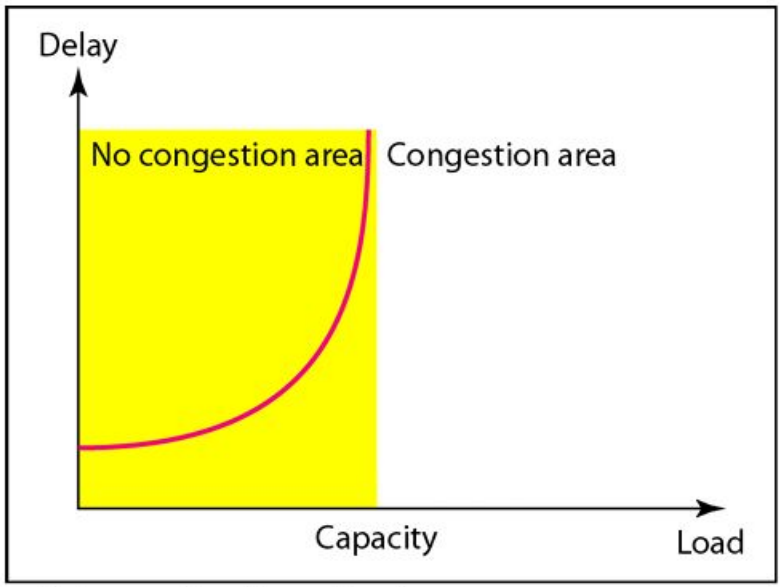
IP addresses versus port numbers



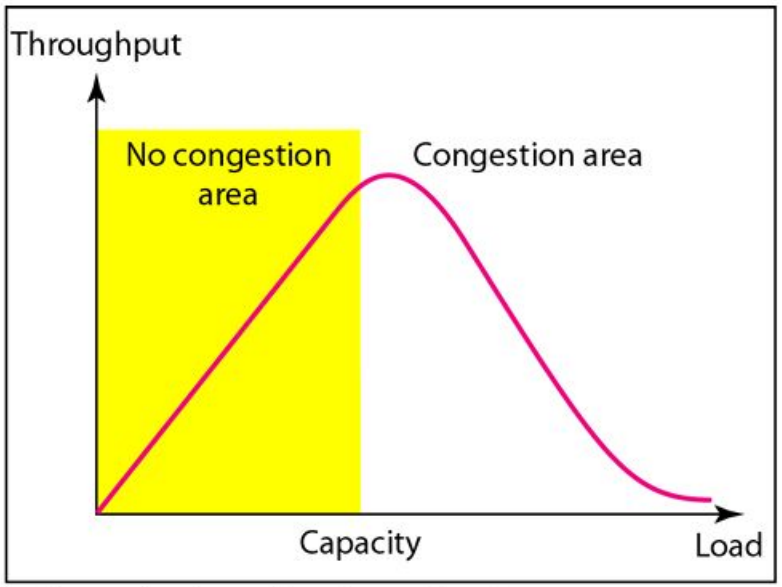
CONGESTION

Congestion in a network may occur if the load on the network—the number of packets sent to the network—is greater than the capacity of the network—the number of packets a network can handle. Congestion control refers to the mechanisms and techniques to control the congestion and keep the load below the capacity

CONGESTION



a. Delay as a function of load



b. Throughput as a function of load

Figure: Packet delay and throughput as functions of load

CONGESTION CONTROL

- A process on the local host, called a client, needs services from a process usually on the remote host, called a server.
- Both processes (client and server) have the same name.
- For communication, we must define the following:

  1. Local host address(logical, physical)
  2. Local process name/id
  3. Remote host address(logical ,physical)
  4. Remote process name/id

Thus we need to use different levels of addressing.

CONGESTION CONTROL

Congestion control refers to techniques and mechanisms that can either prevent congestion, before it happens, or remove congestion, after it has happened. In general, we can divide congestion control mechanisms into two broad categories: open loop congestion control (prevention) and closed-loop congestion control (removal)

CONGESTION CONTROL

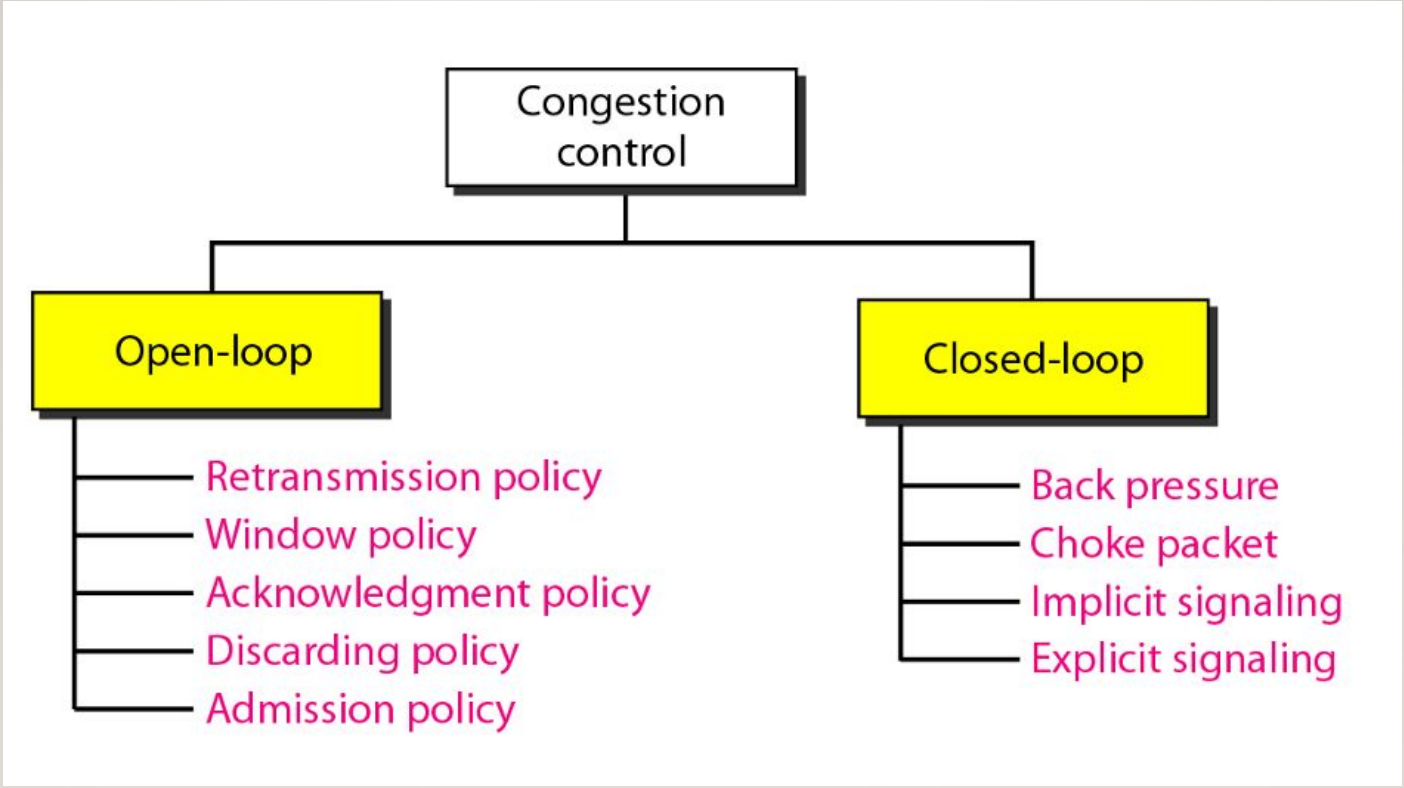


Figure: Congestion control categories

OPEN-LOOP CONGESTION CONTROL

In open-loop congestion control, policies are applied to prevent congestion before it happens. In these mechanisms, congestion control is handled by either the source or the destination. We give a brief list of policies that can prevent congestion.

1.Retransmission Policy

Retransmission is sometimes unavoidable. If the sender feels that a sent packet is lost or corrupted, the packet needs to be retransmitted. Retransmission in general may increase congestion in the network. However, a good retransmission policy can prevent congestion. The retransmission policy and the retransmission timers must be designed to optimize efficiency and at the same time prevent congestion. For example, the retransmission policy used by TCP is designed to prevent or alleviate congestion.

OPEN-LOOP CONGESTION CONTROL

2. Window Policy

The type of window at the sender may also affect congestion. The Selective Repeat window is better than the Go-Back-N window for congestion control. In the Go-Back-N window, when the timer for a packet times out, several packets may be resent, although some may have arrived safe and sound at the receiver. This duplication may make the congestion worse. The Selective Repeat window, on the other hand, tries to send the specific packets that have been lost or corrupted.

3. Acknowledgment Policy

The acknowledgment policy imposed by the receiver may also affect congestion. If the receiver does not acknowledge every packet it receives, it may slow down the sender and help prevent congestion. Several approaches are used in this case. A receiver may send an acknowledgment only if it has a packet to be sent or a special timer expires. A receiver may decide to acknowledge only N packets at a time. We need to know that the acknowledgments are also part of the load in a network. Sending fewer acknowledgments means imposing less load on the network.

OPEN-LOOP CONGESTION CONTROL

4. Discarding Policy

A good discarding policy by the routers may prevent congestion and at the same time may not harm the integrity of the transmission. For example, in audio transmission, if the policy is to discard less sensitive packets when congestion is likely to happen, the quality of sound is still preserved and congestion is prevented or alleviated.

5. Admission Policy

An admission policy, which is a quality-of-service mechanism, can also prevent congestion in virtual-circuit networks. Switches in a flow first check the resource requirement of a flow before admitting it to the network. A router can deny establishing a virtual circuit connection if there is congestion in the network or if there is a possibility of future congestion.

CLOSED-LOOP CONGESTION CONTROL

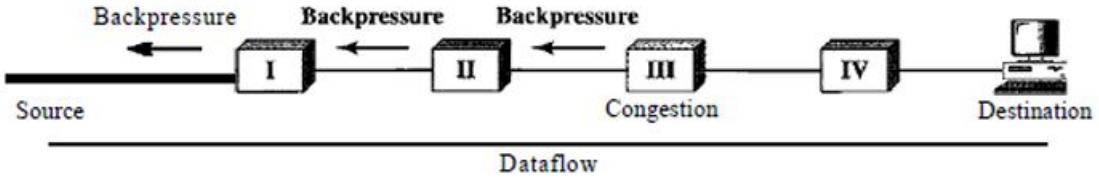
Closed-loop congestion control mechanisms try to alleviate congestion after it happens. Several mechanisms have been used by different protocols. We describe a few of them here.

1.Backpressure

The technique of backpressure refers to a congestion control mechanism in which a congested node stops receiving data from the immediate upstream node or nodes. This may cause the upstream node or nodes to become congested, and they, in turn, reject data from their upstream nodes or nodes. And so on. Backpressure is a node-to-node congestion control that starts with a node and propagates, in the opposite direction of data flow, to the source. The backpressure technique can be applied only to virtual circuit networks, in which each node knows the upstream node from which a flow of data is corning. Following figure shows the idea of backpressure.

CLOSED-LOOP CONGESTION CONTROL

Backpressure method for alleviating congestion:



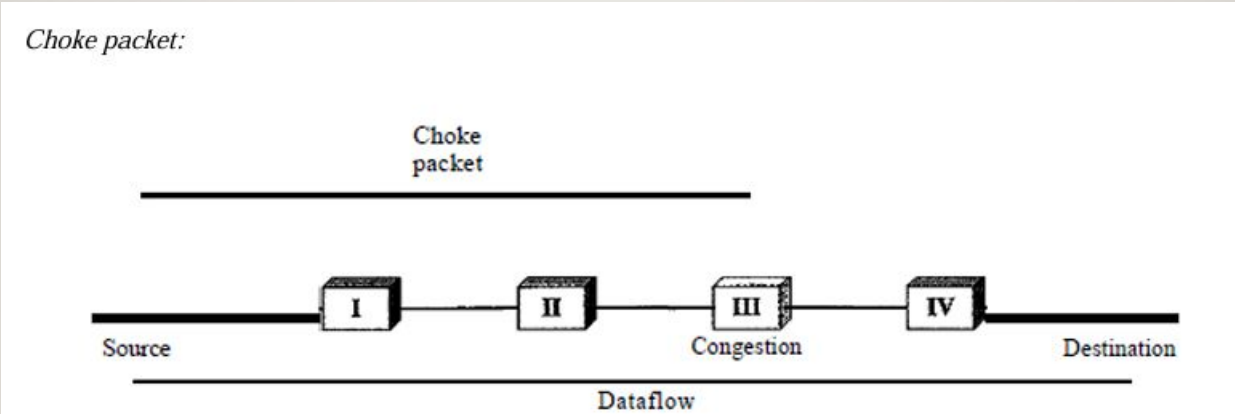
Node III in the figure has more input data than it can handle. It drops some packets in its input buffer and informs node II to slow down. Node II, in turn, may be congested because it is slowing down the output flow of data. If node II is congested, it informs node I to slow down, which in turn may create congestion. If so, node I informs the source of data to slow down. This, in time, alleviates the congestion. Note that the pressure on node III is moved backward to the source to remove the congestion.

CLOSED-LOOP CONGESTION CONTROL

2. Choke Packet

A choke packet is a packet sent by a node to the source to inform it of congestion. Note the difference between the backpressure and choke packet methods. In backpressure, the warning is from one node to its upstream node, although the warning may eventually reach the source station. In the choke packet method, the warning is from the router, which has encountered congestion, to the source station directly. The intermediate nodes through which the packet has travelled are not warned. We have seen an example of this type of control in ICMP. When a router in the Internet is overwhelmed with IP datagrams, it may discard some of them; but it informs the source host, using a source quench ICMP message. The warning message goes directly to the source station; the intermediate routers, and does not take any action. Following figure shows the idea of a choke packet.

CLOSED-LOOP CONGESTION CONTROL



CLOSED-LOOP CONGESTION CONTROL

3. Implicit Signalling

In implicit signalling, there is no communication between the congested node or nodes and the source. The source guesses that there is a congestion somewhere in the network from other symptoms. For example, when a source sends several packets and there is no acknowledgment for a while, one assumption is that the network is congested. The delay in receiving an acknowledgment is interpreted as congestion in the network; the source should slow down. We will see this type of signalling when we discuss TCP congestion control later in the chapter.

4.Explicit Signalling

The node that experiences congestion can explicitly send a signal to the source or destination. The explicit signalling method, however, is different from the choke packet method. In the choke packet method, a separate packet is used for this purpose; in the explicit signalling method, the signal is included in the packets that carry data. Explicit signalling, as we will see in Frame Relay congestion control, can occur in either the forward or the backward direction.

CLOSED-LOOP CONGESTION CONTROL

5. Backward Signalling

A bit can be set in a packet moving in the direction opposite to the congestion. This bit can warn the source that there is congestion and that it needs to slow down to avoid the discarding of packets.

6. Forward Signalling

A bit can be set in a packet moving in the direction of the congestion. This bit can warn the destination that there is congestion. The receiver in this case can use policies, such as slowing down the acknowledgments, to alleviate the congestion.

UDP—USER DATAGRAM PROTOCOL

- UDP is an unreliable connectionless transport protocol,
- No connection is established .
- Provide Best effort service
- Does not provide flow and error control
- Does multiplexing and demultiplexing of data from multiple processes.
- Require minimum overhead.
- communication is much faster.

UDP—USER DATAGRAM PROTOCOL

- The Internet protocol suite supports a connectionless transport protocol called UDP (User Datagram Protocol).
- UDP provides a way for applications to send encapsulated IP datagrams without having to establish a connection. UDP is described in RFC 768.
- UDP transmits segments consisting of an 8-byte header followed by the payload. The header is shown in Fig. 4.16. The two ports serve to identify the endpoints within the source and destination machines.
- When a UDP packet arrives, its payload is handed to the process attached to the destination port. This attachment occurs when the BIND primitive or something similar is used.

UDP SERVICES

- **Process-to-Process Communication:** UDP provides process-to-process communication using socket addresses, a combination of IP addresses and port numbers.
- **Connectionless Services:** As mentioned previously, UDP provides a connection less service. This means that each user datagram sent by UDP is an independent datagram. There is no relationship between the different user data grams even if they are coming from the same source process and going to the same destination program.
- **Flow Control:** UDP is a very simple protocol. There is no flow control, and hence no window mechanism. The receiver may overflow with incoming messages.
- **Error Control:** There is no error control mechanism in UDP except for the checksum. This means that the sender does not know if a message has been lost or duplicated.
- **Checksum:** UDP checksum calculation includes three sections: a pseudo header, the UDP header, and the data coming from the application layer. The pseudo header is the part of the header of the IP packet in which the user datagram is to be encapsulated with some fields filled with 0s

UDP APPLICATIONS

- **UDP Features**
- **Connectionless Service**

As we mentioned previously,

UDP is a connectionless protocol. Each UDP packet is independent from other packets sent by the same application program. This feature can be considered as an advantage or disadvantage depending on the application requirements.

UDP does not provide error control; it provides an unreliable service. Most applications expect reliable service from a transport-layer protocol. Although a reliable service is desirable.

UDP APPLICATIONS

- **UDP Features**
- **Typical Applications**

The following shows some typical applications that can benefit more from the services of UDP

- UDP is suitable for a process that requires simple request-response communication with little concern for flow and error control
- UDP is suitable for a process with internal flow- and error-control mechanisms. For example, the Trivial File Transfer Protocol (TFIP)
- UDP is a suitable transport protocol for multicasting. Multicasting capability is embedded in the UDP software
- UDP is used for management processes such as SNMP
- UDP is used for some route updating protocols such as Routing Information Protocol (RIP)
- UDP is normally used for interactive real-time applications that cannot tolerate uneven delay between sections of a received message

Transmission Control Protocol (TCP)

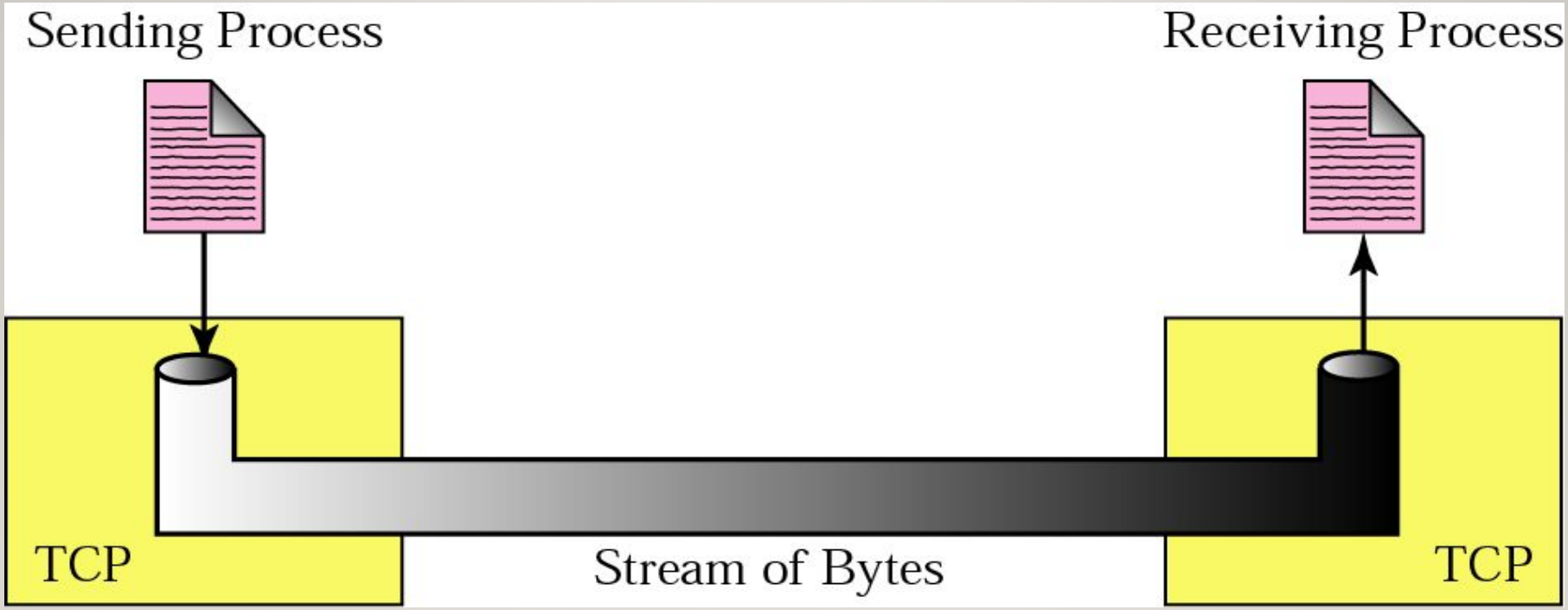
- Full duplex Communication
- Connection–oriented service
- Reliable service
- TCP is a connection oriented protocol; it creates a virtual connection between two TCPs to send data.
- In addition, TCP uses flow and error control mechanisms at the transport level.

Applications of TCP

- Application requiring reliable ,full duplex communication to send bulk data like file transfer use TCP.
- Mail protocols like SMTP and POP3 use TCP.
- TCP is used for HTTP client and HTTP sever communication.
- Some transport level security protocols like TLS uses TCP for transaction on the internet.

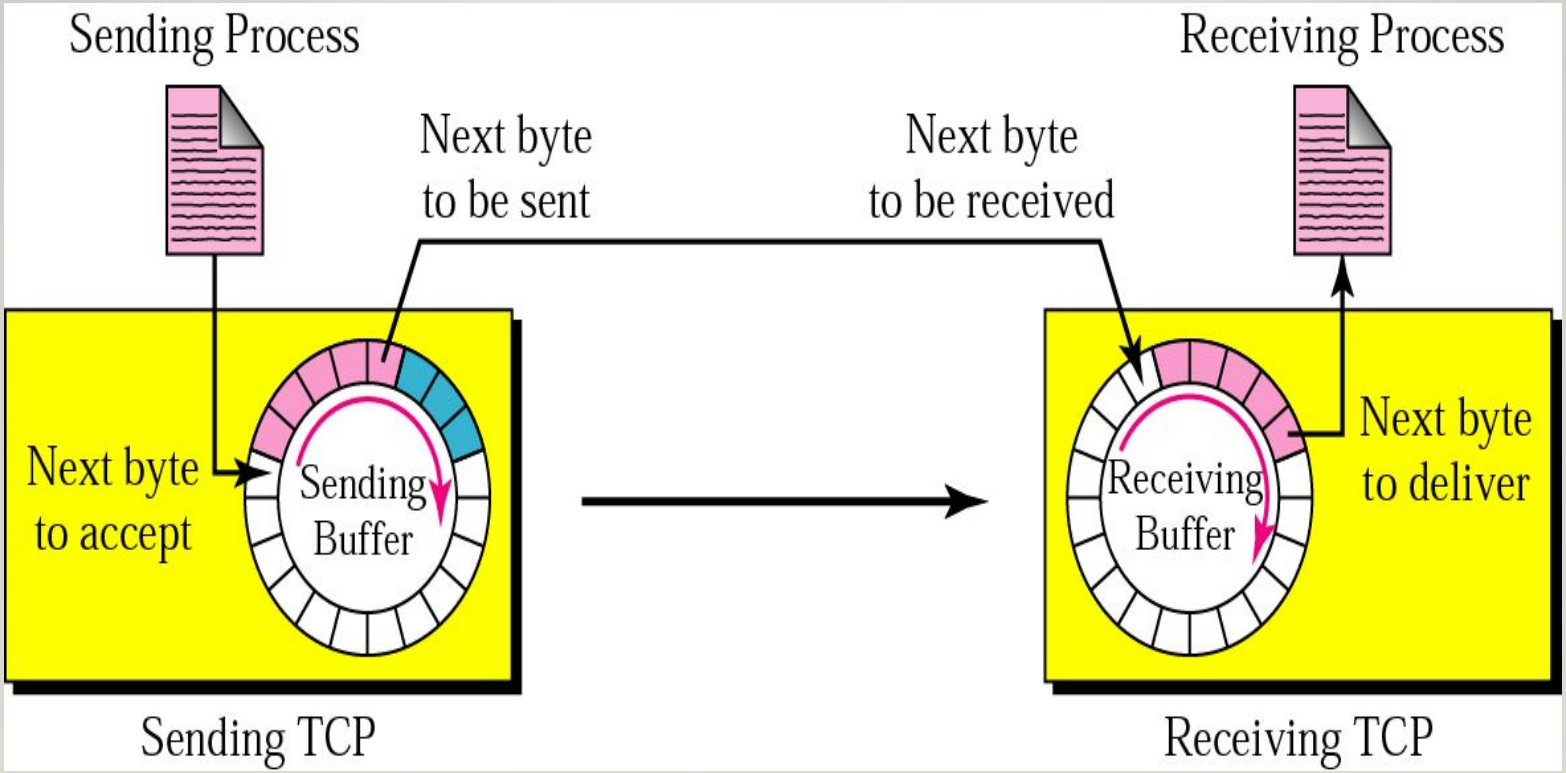
TCP Services

Stream delivery

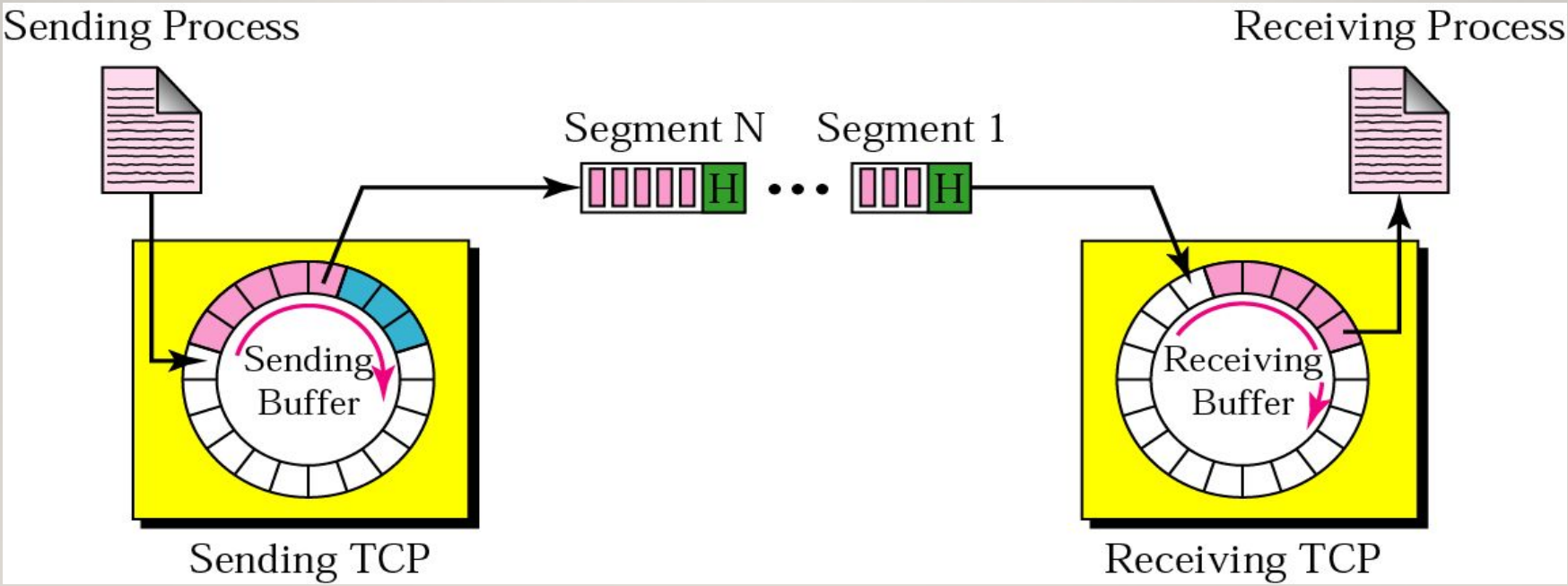


TCP Services

Sending and receiving buffers



TCP segments



TCP Features

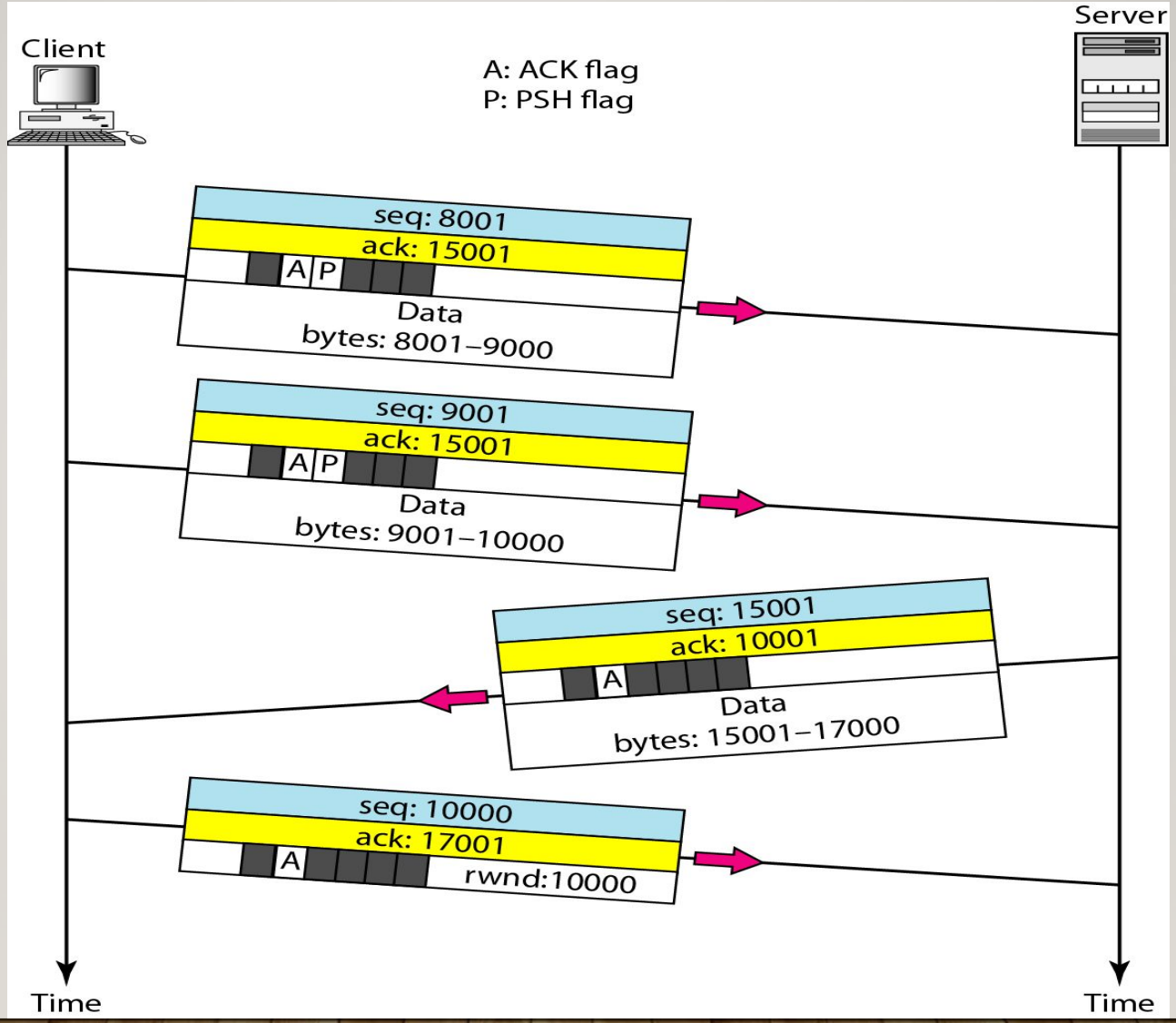
Numbering System :

Byte Number : TCP numbers all data bytes that are transmitted in a connection. Numbering is independent in each direction. When TCP receives bytes of data from a process, it stores them in the sending buffer and numbers them. The numbering does not necessarily start from . Instead, TCP generates a random number between 0 and 232 - 1 for the number of the first byte.

Sequence Number : After the bytes have been numbered, TCP assigns a sequence number to each segment that is being sent. The sequence number for each segment is
the number of the first byte carried in that segment.

Acknowledgment Number : Each party also uses an acknowledgment number to confirm the bytes it has received. However, the acknowledgment number defines the number of the next byte that the party expects to receive.

Data transfer



Port Number	Description
1	TCP Port Service Multiplexer (TCPMUX)
7	ECHO
18	Message Send Protocol (MSP)
20	FTP -- Data
21	FTP -- Control
22	SSH Remote Login Protocol
23	Telnet
25	Simple Mail Transfer Protocol (SMTP)
53	Domain Name System (DNS)
69	Trivial File Transfer Protocol (TFTP)
70	Gopher Services
79	Finger
80	HTTP
103	X.400 Standard
108	SNA Gateway Access Server
109	POP2
110	POP3
115	Simple File Transfer Protocol (SFTP)

TCP Performance Issues

- High Overhead: The 20-byte header and connection-establishment phases delay initial data transmission.
- Congestion Control: During heavy network traffic, TCP deliberately slows down transmission rates, drastically reducing throughput.
- Head-of-Line Blocking: Because TCP enforces strict sequential delivery, all subsequent packets must wait if a single packet in the sequence is lost or delayed.

UDP Performance Issues

- Lack of Flow Control: Because UDP provides no feedback to the sender, a fast sender can easily overwhelm and cause packet loss at a slower receiver.
- Unfairness to TCP: Because UDP does not back off or throttle its transmission rate during network congestion, it can "starve" competing TCP streams of bandwidth.
- Error Handling: The protocol requires the application layer to implement its own mechanisms for data verification, re-sequencing, and retransmissions.